

AlertMind

AI-Assisted Mini Security Operations Centre (SOC)

AlertMind pairs a conventional, rule-based SIEM (Wazuh) with a guarded AI triage layer to help SOC analysts move from "alert received" to "informed decision" faster — without ever taking detection or response authority away from the human analyst.

Capstone project — IIT Roorkee × Futureense, PG Certificate in AI and GenAI-Powered Cybersecurity
Team **Brain Bytes**: [Indrajit Singh](#) · Jay Goyal · Vansh Gupta

Table of Contents

- [Overview](#)
- [Why AlertMind](#)
- [Architecture](#)
- [AI Processing Pipeline](#)
- [Features](#)
- [Custom Detection Rules](#)
- [Tech Stack](#)

- [Project Structure](#)
 - [Getting Started](#)
 - [Dashboards](#)
 - [Guardrails & Responsible AI](#)
 - [Incident Response Playbook](#)
 - [Workflow Efficiency \(MTTD / MTTR\)](#)
 - [Roadmap](#)
 - [Disclaimer](#)
 - [Team](#)
-

Overview

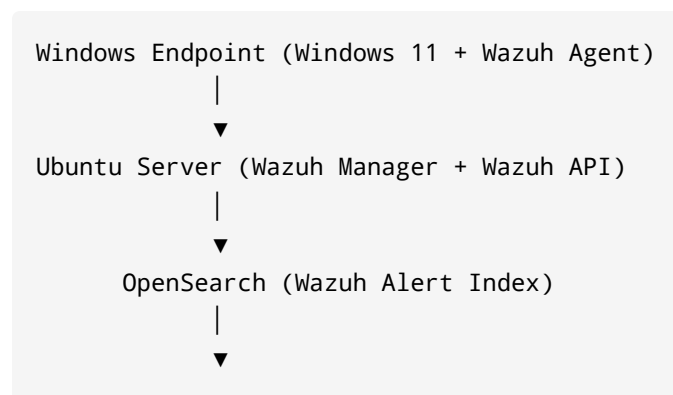
A SOC analyst's biggest bottleneck usually isn't detection — it's everything that happens *after* an alert fires: reading raw JSON, working out the MITRE ATT&CK technique, and finding the right playbook step. AlertMind targets exactly that gap.

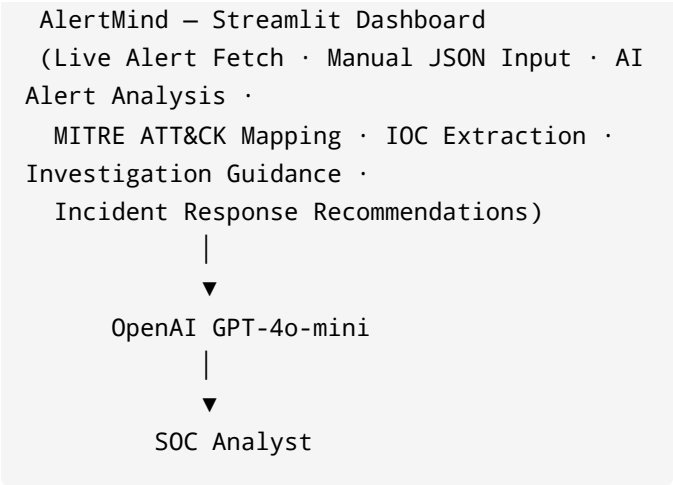
Wazuh does all the detection, exactly as it would in any standard deployment. AlertMind sits downstream, reads the alert once it already exists, and hands the analyst a structured summary — technique mapping, severity, IOCs, investigation steps, and a recommended response — generated by an LLM under a set of validation and sanitization guardrails. The analyst reviews, validates, and acts. AlertMind never acts on its own.

Why AlertMind

- **Detection stays where it belongs.** Wazuh's rule engine is unchanged and fully independent of the AI layer.
- **The AI is a triage assistant, not an autonomous agent.** No auto-containment, no auto-blocking, no unsupervised actions.
- **Every alert is sanitized before it reaches an external model.** Size limits, schema validation, prompt-injection screening, and sensitive-field redaction all happen first.
- **Structured output, not a chat transcript.** The model returns fixed, labeled fields — summary, MITRE mapping, severity, IOCs, investigation steps, response actions — so results are consistent and auditable.

Architecture





Full diagram: [docs/AlertMind Architecture.svg](#)

Component	Role
Wazuh Agent	Collects endpoint events
Wazuh Manager	Processes security events and applies detection rules
OpenSearch	Stores and indexes alerts
AlertMind	Retrieves alerts and performs AI-assisted analysis
OpenAI GPT-4o-mini	Generates structured security intelligence
SOC Analyst	Validates findings and performs incident response

AI Processing Pipeline

Every alert passes through a validation-first pipeline before it ever reaches the model:

```
Wazuh Alert JSON
→ parse_wazuh_alert()
→ Guardrails Engine
  • Payload Size Validation
  • Wazuh Schema Validation
  • Prompt Injection Detection
  • Sensitive Data Redaction
→ Sanitized Alert Payload
→ OpenAI GPT-4o-mini
→ Structured JSON Response
  • AI Summary
  • MITRE ATT&CK Mapping
  • Severity Classification
  • IOC Extraction
  • Investigation Steps
  • Incident Response Actions
→ AlertMind Streamlit Dashboard
→ SOC Analyst
```

Full diagram: [docs/AlertMind AI Pipeline.svg](#)

AlertMind validates and sanitizes every alert before AI analysis. The AI acts as a decision-support assistant while the SOC analyst retains full control over investigation and response.

Features

-  **Live alert ingestion** directly from OpenSearch, or manual JSON paste for ad-hoc analysis
-  **AI-generated executive summaries** of raw Wazuh alerts, in plain language
-  **Automatic MITRE ATT&CK technique mapping**
-  **Severity classification** to help prioritize triage order
-  **IOC extraction** from alert data
-  **Investigation guidance** and next-step recommendations
-  **Incident Response Playbook integration** — AI recommendations link back to documented response procedures
-  **Guardrails Engine** — payload validation, schema checks, prompt-injection detection, and sensitive-data redaction on every request

Custom Detection Rules

Thirteen custom Wazuh rules (100101–100113) covering system enumeration, network discovery, and suspicious command execution:

Rule ID	Command	Description
100101	whoami	User Information Discovery
100102	ipconfig	Network Configuration Discovery
100103	hostname	System Information Discovery
100107	net localgroup	Local Group Discovery
100108	netstat	Network Connections Discovery
100109	ping	Network Discovery
100110	PowerShell	PowerShell Execution
100111	cmd	Command Prompt Execution
100112	certutil	CertUtil Execution (LOLBin abuse)
100113	nslookup	DNS Lookup

Each rule is mapped to its corresponding MITRE ATT&CK technique and a documented response

procedure in the [Incident Response Playbook](#).

Tech Stack

Layer	Technology
Endpoint	Windows 10/11, Sysmon, Wazuh Agent
Detection	Wazuh Manager, Wazuh API (Ubuntu)
Storage / Index	OpenSearch
Application	Python, Streamlit
AI Analysis	OpenAI GPT-4o-mini

Project Structure

```
AlertMind/  
├─ app.py                # Streamlit  
entry point  
├─ analyzer.py           # AI analysis  
orchestration  
├─ guardrails.py         # Payload  
validation, schema checks, redaction  
├─ prompts.py            # Prompt  
templates for the AI Assistant
```



```
├─ styles.py                # Dashboard
styling
├─ utils.py                 # Shared
helper functions
├─ opensearch_client.py     # OpenSearch
alert retrieval
├─ wazuh_api.py             # Wazuh
Manager REST API client
├─ docs/
│   └─ AlertMind_Architecture.svg
│   └─ AlertMind_AI_Pipeline.svg
```

Getting Started

Prerequisites

- A running Wazuh Manager + OpenSearch deployment, with at least one monitored endpoint
- Python 3.10+
- An OpenAI API key with access to `gpt-4o-mini`

Setup

```
git clone
https://github.com/indrajitisingh/AlertMind
.git
cd AlertMind
pip install -r requirements.txt
```

Configure your Wazuh API and OpenAI credentials as environment variables (see `utils.py` for the

expected variable names), then launch the dashboard:

```
streamlit run app.py
```

⚠ The demo build ships with default local credentials for evaluator convenience; override them via environment variables before any non-local use, and rotate any default secrets before deployment.

Dashboards

AlertMind's own AI-assisted view sits alongside standard Wazuh operational dashboards used for day-to-day monitoring:

- **Daily SOC Briefing** — top triggered rules, top agents, total alert volume
- **MITRE ATT&CK Dashboard** — top techniques, top tactics, MITRE-mapped alert trends
- **Endpoint Summary** — per-agent event evolution, MITRE tactic breakdown, compliance posture (PCI DSS)

Guardrails & Responsible AI

AlertMind is built around a decision-support model of AI integration, not autonomous response:

- Wazuh performs all detection – the AI model never sees raw endpoint telemetry until an alert already exists
- Every alert is validated and sanitized by the Guardrails Engine before submission to the AI model
- The model is constrained to structured, bounded JSON output across six defined fields
- All AI output is explicitly presented as a *recommendation* for analyst review – never as an automated action
- AlertMind does not trigger containment, blocking, or remediation on its own

Incident Response Playbook

A dedicated, rule-mapped Incident Response Playbook covers all three detection categories (System Enumeration, Network Discovery, Suspicious Command Execution), each following a standard seven-part lifecycle: Overview, Detection, Analysis, Containment, Eradication, Recovery, Lessons Learned.

 [docs/IR_Playbook.docx](#)

Workflow Efficiency (MTTD / MTTR)

A representative, lab-based comparison of manual vs. AI-assisted triage:

Metric	Without AI	With AlertMind AI
MTTD	Near real-time (seconds to ~1 min)	Near real-time (seconds to ~1 min) – unaffected, since Wazuh detects independently
MTTR	~4–6 min (representative estimate)	~1–1.5 min (representative measurement)

These are illustrative workflow measurements from a controlled lab environment, not a formal benchmark study. Full methodology and limitations:

[docs/MTTD MTTR Analysis.md](#)

Roadmap

- [] Expand rule coverage to lateral movement, persistence, and exfiltration techniques
- [] Background alert polling and automatic JWT token refresh
- [] Per-analyst audit logging of AI-assisted decisions
- [] Rate limiting on the Guardrails Engine

Disclaimer

AlertMind is a capstone/educational project. It is not a production-hardened security product. The AI Assistant's output is a recommendation only and must be validated by a qualified analyst before any response action is taken.

Team

Built by **Team Brain Bytes** as part of the IIT Roorkee × FutureNSE PG Certificate Programme in AI and GenAI-Powered Cybersecurity.

Name	Focus Area
Indrajit Singh	Platform architecture, Wazuh/Sysmon/OpenSearch integration, dashboard development
Jay Goyal	Detection rule engineering, testing & validation, Incident Response Playbook
Vansh Gupta	AI integration, guardrails engineering, documentation